The Refined Plan

The one question we're answering: how good is an LLM agent at finding and exploiting memory-safety bugs — and does a hardened allocator (Scudo + MTE) actually stop it?

The whole philosophy in one line: start on tiny things we fully control, prove our measuring tools work there, *then* point the agent at real software. Every phase produces one checkable thing (an "artifact"), and we don't move on until it holds up. This is what keeps us from ever claiming "the agent beat MTE" when the truth was a bug in our own setup.

Word bank (terms used in this plan)

Term	Plain meaning
Agent	The LLM hooked up to tools (run code, read files, read crash logs) so it can actually try things, not just chat.
Harness	The scaffolding we build around the agent: runs the target, feeds it inputs, logs everything.
Oracle / success-checker	An *independent* piece of code that decides "did the attack actually work?" — so the agent can't grade its own homework.
Baseline	A dumb comparison method (random guessing, a normal fuzzer). The agent has to beat these to be interesting.
Fuzzer	A tool that throws tons of random/mutated inputs at a program to find crashes. The standard automated bug-finder.
Threat model	The written rules of the game: what the attacker starts with, what counts as winning.
SYNC / ASYNC	MTE's two modes. SYNC crashes instantly on a bad access (what we want); ASYNC crashes a moment later (leaves a gap).

Term	Plain meaning
Seed	The random starting value. "Fresh seed" = re-roll the randomness, to check a win wasn't a fluke.

The guiding principle

We are **not** trying to break MTE. We're **measuring** whether an LLM agent changes the **cost, reliability, or range** of exploiting a precisely-defined defense. Every phase is built so a result can be stated honestly: what the agent was given, what it achieved, and which assumption it leaned on.

Phase 0 — Prove the ruler works (the first thing we build)

A one-page rules sheet, plus a few tiny buggy programs that let us confirm MTE behaves as advertised before any agent is involved.

- Write the threat model: what the attacker starts with, what counts as success, exact setup (device/emulator, Android build, Scudo version + commit, tag policy, SYNC vs ASYNC).
- Write 3–5 tiny C programs with deliberate bugs — one linear overflow, one use-after-free, one tiny intra-granule overflow, one write to an untagged global — each with a known expected outcome (which ones MTE *should* catch, which slip through).
- Build the success-checker and logging **before** the agent exists.

Artifact: the rules sheet, the buggy programs, and a passing test showing valid runs work and the bugs fault exactly as expected.

Phase 1 — Measure the dumb baselines

Get numbers for non-agent methods, so the agent has something to beat.

- Run a random-guess baseline and a normal fuzzer against those same tiny programs.
- Record three rates: did it reach the bug, did it get past MTE, did it achieve a real effect — with totals and costs.
- This runs fine on a laptop + emulator; mark anything needing real phone hardware as a later confirmation.

Artifact: a baseline table with error bars (so a lucky run isn't mistaken for a reliable one).

Phase 2 — Bring in the agent (still on the tiny programs)

- Give the agent the same programs under the same rules; vary what it starts with (source code, just the binary, crash logs) to see what help matters.
- The independent success-checker decides wins. Replay every claimed win from a clean start with a fresh seed.
- When it wins, label *why*: lucky collision, leaked the tag, swapped a pointer, found an allocator gap, hit untagged memory, or beat the async window.

Artifact: the first real comparison — agent vs baselines, broken down by how much help it got.

Phase 3 — Can it redo a known attack?

- Pick one published attack (an easy tag-collision first; a TikTag-style side channel is harder and needs real hardware).
- Measure whether the agent can reproduce it within a budget.
- Keep this **separate** from brand-new tasks, because published attacks may be sitting in the model's training data (so reproducing one proves less than discovering one).

Artifact: a short report stating plainly — did it re-run a known trick, or find something new?

Phase 4 — The real targets (a read, then a write)

We use **two** real targets, in order (see the Target tab for full detail):

1. **dav1d / CVE-2025-48592 — the READ target first.** Easiest to build, so it de-risks the harness. Reason first about whether the bug even survives MTE (likely it becomes a crash — a fine result), then run the agent.
2. **Skia / CVE-2025-32318 — the WRITE target second,** once the machinery is proven. A write gives a clearer "took control" success, exercising the path MTE is really built to stop.

For each: reason about how the bug interacts with MTE, then run the agent under the full rules and report all three rates with costs and error bars. **Admit each target's drawback** (dav1d's win is weak; Skia is hard to build and nearer MTE's soft end) — that honesty is part of the result.

Artifact: the end-to-end case studies.

What we need, and what's still open

Need	For	Status
MTE-capable phone (Pixel 8+)	Phases 3–4, any side-channel work	Expected — confirm
Berkeley API keys for security testing	Running the agent (Phases 2–4)	Confirm access
Laptop + emulator	Phases 0–2	Have it now
Exact Scudo version + MTE mode from Marius	Pinning Phase 0	Open (ask him)
Chosen targets	Phase 4	Done — dav1d (read) + Skia (write)

What to send Marius

The short list of open questions: exact Scudo version and Android build; SYNC or ASYNC; are side channels in scope; what exactly counts as "success"; confirm phone + API-key access; and — new — **one target done deeply, or commit to both the read and write target now?** Plus: how is the Skia bug reached, remotely or after a foothold?